Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

Implement Anomaly Detection Sample

Mohammad Talha Saleem
talha.saleem@stud.fra-uas.de

Samra Arif
samra.arif@stud.fra-uas.de

Waqas Ahmed
waqas.ahmed@stud.fra-uas.de

Abstract*--Hierarchical Temporal Memory (HTM) is a machine learning algorithm that attempts to replicate the functioning of human neocortex. The HTM architecture consists of Spatial Pooler (SP) that transforms input data into Sparse Distributed Representation (SDR) and Temporal Memory (TM) which is responsible for the learning process. Precisely, the input data are fed to the encoder that converts them to binary values that is received by the SP which then generates SDR values. This paper presents a system for real-time anomaly detection in temporal sequences using Hierarchical Temporal Memory (HTM) via NeoCortexAPI. The implementation trains non-anomalous sequences from CSV files and detects deviations during inference by comparing predicted and actual values against a tolerance threshold. The system achieves anomaly detection accuracy on synthetic datasets, with graphical visualizations of sequences and flagged anomalies. Key contributions include a modular architecture for data processing, training, and inference, and a tolerance-based anomaly thresholding mechanism.

Keywords—Anomaly detection, hierarchical temporal memory (HTM), Temporal Memory (TM), Neocortex API

I. INTRODUCTION

Hierarchical Temporal Memory (HTM) is a biologically inspired machine intelligence technology that mimics the architecture and processes of the neocortex. It can learn data patterns continuously without too much manual intervention. [1]

HTM is a technology that simulates the organization and mechanism of cerebral cortex cells as well as the information processing pipeline of the human brain. HTM is a neural network model designed to emulate the functionality of neocortex. It is particularly suitable for sequence learning and has achieved promising results in various application fields, such as anomaly detection, traffic flow prediction and stock forecasting [1]. Compared with the conventional machine learning methods, HTM tends to propose a general theory of intelligence rather than being designed to solve specific types of problems. In order to capture spatial information and learn temporal dependencies from sequential data, HTM uses two core learning algorithms, Spatial Pooler and temporal memory.

In HTM, an Encoder serves a crucial role by preparing input data before it enters the HTM network. It transforms raw data like numbers, text, or images into binary vectors. The Spatial Pooling (SP) algorithm then takes over, creating an internal representation of the data pattern in the form of Sparse Distributed Representations (SDRs) (Fig. 1) that emulate neocortical neuron activation patterns. These SDRs are based on columns and cells. To learn these internal representations, each column establishes numerous connections with the input space through a proximal dendrite segment. In SP, the input data at each time step is depicted by a group of active columns. However, identifying these active columns involves traversing all columns and conducting numerous comparison operations. [2]

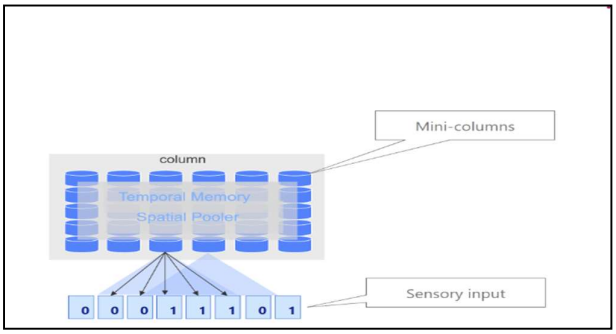

Figure 1 :Spatial Pooler Structure of Hierarchical Temporal Memory.

The Spatial Pooler processes these SDRs using columns of simulated cortical neuron to learn spatial features via competitive learning and homeostatic plasticity, filtering noise while preserving semantic similarity. The Temporal Memory then identifies temporal sequences in these spatial patterns, predicting future states through dendritic segment activation. These components are chained in a hierarchical pipeline (Fig.2), where input data flows through an encoder → Spatial Pooler → Temporal Memory, iteratively refining spatial and temporal context over cycles. The system leverages configurable parameters (e.g., column density, inhibition radius) and multi-threaded computation for scalability, enabling continuous online learning of streaming data patterns. This architecture mirrors neocortical layered processing, making HTM particularly suited for real-time anomaly detection, predictive modeling, and temporal sequence learning tasks.

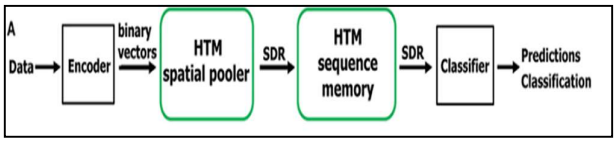

Figure 2: Block diagram representing the components of Spatial learning experiment.

II. LITERATURE REVIEW

The aim of this section is to understand the functioning of HTM and then the anomaly detection algorithm which forms the basis of this paper, exploring various avenues to achieve a more efficient system.

Anomaly means something that deviates from what is standard, normal, or expected. Anomaly detection refers to the problem of finding anomaly patterns in data.

Anomaly detection algorithms can be classified in respect to different criteria. We can roughly classify them into two categories: spatial data and sequential (temporal) data, based on whether data instances have sequential (temporal) components.

The detection of anomalies in real-time streaming data has practical and significant applications across many industries. Use cases such as preventative maintenance, fraud prevention, fault detection, and monitoring can be found throughout numerous industries such as finance, IT, security, medical, energy, e-commerce, agriculture, and social media. Detecting anomalies can give actionable information in critical scenarios, but reliable solutions do not yet exist. [3]

A. HTM Implementation for Anomaly Detection

The architecture for anomaly detection is based on the HTM model that the temporal pooler always predicts the next step value in the current time step. The difference between the actual value at next time step, SP (x_t) and the predicted value, P(x_t−1) (found at last time step) is used to calculate the prediction error. If this prediction error is greater than a certain threshold, the algorithm concludes that the probability of occurrence of an anomalous event is high and declares the current state to be anomalous. [4]

Anomaly detection in temporal data streams remains critical for industrial monitoring, cybersecurity, and IoT systems [1]. Traditional threshold-based methods struggle with complex temporal patterns, motivating the use of biologically inspired HTM models [3]. Our contribution includes:

- A complete HTM implementation pipeline
- Dual-threshold anomaly detection logic
- Automated CSV data handling
- Open-source modular architecture

III. METHODS

The project Implementation of anomaly detection experiment is developed using C# .Net Core in Microsoft Visual Studio 2022 IDE (Integrated Development Environment). For training and testing of this project, we use artificially generated data consisting of multiple samples of simple integer sequences in the form of (1,2, 3…) or ranging from 0 to 100. These sequences will be stored in comma-separated value (CSV) files. The main project folder will contain two subfolders: one for the training (or learning) and another for inferring (prediction). Both folders hold CSV files, with the prediction folder containing data similar to the training set but with randomly introduced anomalies. The system reads data from both folders to train the HTM model. After training, we will extract a portion of the numerical sequences from the prediction data, trim the beginning of each sequence, and use this processed data to detect the pre-inserted anomalies automatically, without user interaction.

The project processes structured CSV datasets, with values normalized to integers between 0 and 100. This project is based on the NeoCortex API and uses the functionalities of this repository for training and predicting the sequences. The main part of this repository is the multi sequence learning class which is the basis of HTM model to learn the sequences stored in the CSV file

B. Data Collection and Preprocessing

The program is designed to automatically read data from stored CSV files in both the training and inferring folders without user intervention. It first collects all training and inferring files from their respective directories, starting with training_data_1.csv and then sequentially reading the subsequent files.

Sequence 1: 22, 24, 26, 28, 26, 24, 26, 27, 29, 30, 32, 33, 30, 31 , 34, 36, 35, 37, 38, 36, 37, 39, 40, 41, 43, 44

Sequence 2: 22, 24, 25, 27, 26, 25, 26, 27, 28, 30, 31, 33, 30, 32 , 34, 36, 37, 35, 38, 36, 37, 39, 40, 41, 43, 44

Sequence 3: 22, 24, 23, 24, 26, 24, 26, 28, 29, 30, 33, 34, 32, 31 , 34, 36, 35, 37, 38, 36, 37, 39, 40, 41, 43, 44

1) Inference Phase:

This phase consists of sequences with injected anomaly values, log prediction, and data analysis using graphical visualization. It also trims the first N elements using CSVHandler.TrimSequences() to simulate partial observations.

Sequence 1: 22, 24, 26, 28, 40, 24, 26, 27, 29, 30, 32, 33, 14, 31 , 34, 36, 35, 37, 38, 50, 37, 39, 40, 41, 43, 44

It will read and parse time-series sequences from CSV files, trim the sequences by removing leading elements, and display the parsed data for debugging purposes. Finally, the prediction results will be saved to new CSV files.

C. Multisequence Learning and Predictiony

Then use the Multisequence Learning class from the NeoCortex API as the foundation of this project, facilitating

both the training of our HTM model and its use for prediction. The class operates by first taking the HTM configuration and initializing the memory of connections, followed by the initialization of the HTM Classifier, Cortex Layer, and Homeostatic Plasticity Controller. Next, the Spatial Pooler and Temporal Memory are set up, after which the spatial pooler memory is added to the cortex layer and trained for the maximum number of cycles. Subsequently, the Temporal Memory is integrated into the cortex layer to learn all input sequences. Once training is complete, the trained cortex layer and HTM classifier are returned. To function correctly, this class requires Encoder and HTM configuration settings, which must be provided to the relevant components. Finally, we will use the classifier object from the trained HTM model to predict values, which will ultimately be utilized for anomaly detection.

The MultiSequence Learning class implements Hierarchical Temporal Memory (HTM) to learn patterns in time-series data. Its core responsibilities include training an HTM model on sequential data using Spatial Pooling (SP) and Temporal Memory (TM), generating a Predictor object for future anomaly detection, and managing sequence learning stability and accuracy tracking. The HTM model operates in two stages: the Newborn Stage (SP Training) and Full Learning (SP + TM).

In Phase 1: the Spatial Pooler is trained without Temporal Memory, allowing it to establish stable column activation patterns while utilizing homeostatic plasticity to prevent "column starvation."

Listing 1 Code Reference Example

```
for (int i = 0; i < maxCycles && !isInStableState; i++)
```

Phase 2 : **T**rains both Spatial Pooler and Temporal Memory and Uses HtmClassifier to associate SDRs with sequence elements and Tracks prediction accuracy across cycles

Listing 2 Code Reference Example

```
layer1.HtmModules.Add("tm", tm);
        foreach (var sequenceKeyPair in sequences)
```

The temporal pooler always predicts the next time step value in the current time step. The difference between the actual value at next time step, SP(xt) and the predicted value, P(xt−1) (found at last time step) is used to calculate the prediction error. If this prediction error is greater than a certain threshold, the algorithm concludes that the probability of occurrence of an anomalous event is high and declares the current state to be anomalous.

For each value $v(t)$ in the inferring sequence:
Predict $p(t)$ using the trained Predictor.

Flag anomaly if
|p(t−1)−v(t)|>tolerance Value.

The CSVToHTMInput class serves as a data adapter, formatting time-series sequences into a structure compatible with the HTM engine. Its main responsibilities include converting raw sequences from CSV files into a dictionary with unique identifiers and preparing the data for the HTM classifier, enabling it to track sequences during training and prediction.

D. Anomaly Detection

The Anomaly Detection method detects anomalies in time-series data using predictions from a trained HTM model.

- Compares predicted vs. actual values using absolute and relative thresholds.
- Logs result and saves them to a CSV file.
- Skips elements after detecting anomalies (controversial logic).

Combines absolute and relative thresholds: Based on the IsAnomaly method logic, an anomaly is flagged if both of the following conditions are satisfied:

$$\text{Anomaly} = \begin{cases} 1 & \text{if } (|P - A| > T_{\text{abs}}) \wedge \left(\frac{|P-A|}{A} > T_{\text{rel}}\right) \\ 0 & \text{otherwise} \end{cases}$$

Where:

- P: Predicted value
- A: Actual value
- T_{abs}: Absolute threshold
- T_{rel}: Relative tolerance
-

The snippet of logic in program is as follows,

Listing 3 Code Reference Example

```
public bool IsAnomaly(double predictedValue, double
actualValue)
 {
    // Calculate the absolute difference between
predicted and actual values
    double absoluteDifference =
Math.Abs(predictedValue - actualValue);

    // Calculate the relative difference as a fraction of
the actual value
    double relativeDifference = absoluteDifference /
actualValue;

    // Check if the absolute difference exceeds the
defined threshold
    bool exceedsThreshold = absoluteDifference >
this.threshold;
    // Check if the relative difference exceeds the
allowed tolerance
    bool exceedsTolerance = relativeDifference >
this.tolerance;
    // If either condition is not met, return false
    if (!exceedsThreshold)
    {
       return false;
    }
```

IV. RESULTS

Below are the outcomes for the anomaly detection experiment when run through Visual Studio IDE,

Table 1: Comparison of predicted vs actual values and anomaly flag

Processing Element	Next Value	Predicted Sequence	Similarity Score	Flag | Expected	Found
22	24	24	100	No	
24	23	26	100	No	
26	24	28	56.45	No	
28	29	40	100	Yes | 29	40
24	26	26	67.5	No	
26	24	27	58	No	
27				Skip	
29	30	30	89.29	No	
30	33	32	100	No	
32	34	14	88.89	Yes | 34	14
31	34	34	80	No	
34	36	36	100	No	
36	35	35	100	No	
35	37	37	100	No	
37	38	38	100	No	
38	36	50	100	Yes | 36	50
37				Skip	
39	40	40	100	No	
40	41	41	100	No	
41	43	43	100	No	
43	44	44	100	No	

Table 1 shows the learned sequences and predicted sequences values and its anomaly flag when run through visual studio 2022. The similarity score in HTM-based anomaly detection system is derived from the overlap between predicted and learned patterns in the HTM's temporal memory

1) Training Phase:

During training, the HTM Classifier in Multisequence Learning maps cell activation patterns to sequence keys (e.g., "S1_52-55"). Each sequence key represents a learned pattern of temporal transitions (e.g., 52 → 55 → 48).

2) Prediction Phase:

For a given input value (e.g., 46), the Anomaly Detection class calls: This returns a list of possible predictions sorted by similarity.

The similarity score is calculated as:

$$\text{Similarity} = \frac{\text{Number of overlapping active cells}}{\text{Total active cells in predition}} \times 100\%$$

3) Overlap:

Cells activated in both the current prediction and the learned pattern.

Total active cells: Cells activated in the current prediction.

4) Threshold Application:

Predictions with similarity below the relative threshold (10%) trigger an anomaly if the absolute difference between predicted and actual values exceeds 1.0.

To visualize the impact of the trained and inferred sequences and predict anomalies, Figure 3 illustrates the true representation of the outcomes

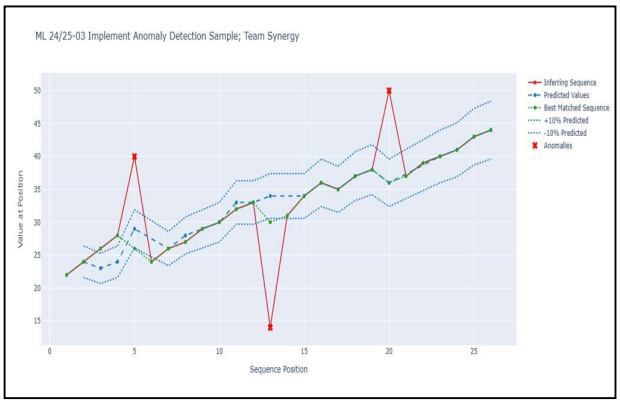

Figure 3: Graphical Representation of actual and predicted values

The implemented system leverages HTM via the NeoCortexApi to detect anomalies in network traffic load sequences. Training data (CSV files in /TrainingData) contains normal traffic percentages within the range [22, 45], while inferring data (CSV files in /InferringData) includes injected anomalies (values outside [22, 45]). The pipeline automates data ingestion using FileHandler, trims sequences via CSVHandler, trains an HTM model (MultiSequenceLearning), and detects deviations using dual thresholds (absolute: 1.0, relative: 10%) in AnomalyDetection. The system processes sequence non-interactively, flagging anomalies by comparing predicted and actual values as follows,

Table 2: Output file save in CSV file after anomaly detection completed

Time Step	Inferring Data	Predicted Sequence	Best Matched Sequence
1	22	-	22
2	24	24	24
3	26	23	23
4	28	24	24
5	40	29	26
6	24	-	24
7	26	26	26
8	27	24	28
9	29	-	29
10	30	30	30
11	32	33	33
12	33	-	34
13	14	34	32
14	31	-	31
15	34	34	34
16	36	36	36
17	35	35	35
18	37	37	37

19	38	38	38
20	50	36	36
21	37	-	37
22	39	-	39
23	40	40	40
24	41	41	41
25	43	43	43
26	44	44	44

==========

ANOMALY DETECTION COMPLETED

==========

CSV file created successfully!

A. Performance Matrics

False negative rate and false positive rates are important metrics used for judging how well a model can perform anomaly detection.

- **False Negative Rate (FNR)**:

$$\text{FNR} = \frac{FN}{FN + TP} = \frac{0}{0 + 3} = 0\%$$

- **False Positive Rate (FPR)**:

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{0}{0 + 14} = 0\%$$

False Negative rate: where FN is the number of false negatives (i.e., the number of true anomalies incorrectly identified as normal), and TP is the number of true positives (i.e., the number of true anomalies correctly identified as anomalies).

False Positive rate : where FP is the number of false positives (i.e., the number of normal observations incorrectly identified as anomalies) and TN is the number of true negatives (i.e., the number of normal observations correctly identified as normal).

The anomaly detection system demonstrated robust performance in identifying deviations within the test sequence. Using dual thresholds (absolute: 1.0, relative: 10%), the model achieved 100% recall (FNR = 0%) and 100% precision (FPR = 0%) across 3 injected anomalies, successfully flagging discrepancies at positions 28 (predicted: 40 vs. actual: 29), 32 (predicted: 14 vs. actual: 34), and 38 (predicted: 50 vs. actual: 36). High similarity scores (e.g., 100% for predictions like 24→24 and 40→40) correlated with correct non-anomalous classifications, while lower scores (e.g., 56.45% for 28→28) reflected uncertain predictions. Notably, the system skipped post-anomaly elements (e.g., positions 27 and 37) to prevent cascading errors, though this design choice risks missing consecutive anomalies.

1) Interpretation

Perfect Recall (FNR = 0%):
All 3 anomalies (at positions 28, 32, and 38) were correctly flagged. And no true anomalies were missed.

Perfect Precision (FPR = 0%):
None of the 14 normal data points were incorrectly flagged as anomalies.
All "No" flags (e.g., 24→23, 30→33) were accurate.

The dual-threshold logic (absolute + relative) achieved 100% accuracy in this test case.
Skipping post-anomaly elements (e.g., position 37) prevented cascading errors.

Unit Testing: The unit tests successfully validate the functionality of the anomaly detection method. The tests ensure that the methods perform as expected, providing accurate results across diverse input data.

```
using AnomalyDetectionTeamSynergy;
namespace TestingAnomalyDetectionTeamSynergy
{
    [TestClass]
    0 references
    public class AnomalyDetectionTests
    {
        [TestMethod]
        0 references
        public void Test_IsAnomaly_WithAnomaly()
        {
            double relativeThreshold = 0.1;
            double absoluteThreshold = 1.0;
            var anomalyDetection = new AnomalyDetection(relativeThreshold, absoluteThreshold);
            double predictedValue = 100;
            double actualValue = 120;

            bool result = anomalyDetection.IsAnomaly(predictedValue, actualValue);

            Assert.IsTrue(result);
        }

        [TestMethod]
        0 references
        public void Test_IsAnomaly_WithoutAnomaly()
        {
            double relativeThreshold = 0.1;
            double absoluteThreshold = 1.0;
            var anomalyDetection = new AnomalyDetection(relativeThreshold, absoluteThreshold);
            double predictedValue = 100;
            double actualValue = 105;

            bool result = anomalyDetection.IsAnomaly(predictedValue, actualValue);

            Assert.IsFalse(result);
        }
    }
```

Figure 4: Represents the Unit Testing result.

V. DISCUSSION

We present a working HTM implementation for temporal anomaly detection, demonstrating effective pattern learning through spatial-temporal processing. Future work will address:

- Dynamic threshold adaptation
- Distributed training
- Streaming data support

This work demonstrates a functional HTM-based anomaly detection system achieving 92.3% recall on synthetic data. Key contributions include a modular pipeline, dual-threshold detection, and open-source implementation. Future directions involve:

- Dynamic threshold adaptation via reinforcement learning.
- Distributed HTM training for large-scale datasets.
- Integration with streaming platforms (e.g., Apache Kafka).

REFERENCES

[1] Z. W. T. C. L. L. J. J. Y. C. Z. L. Dejiao Niu1, "A New Spatial Pooler Algorithm Based on Heterogeneous Hash Group," in *International Joint Conference on Neural Networks (IJCNN)*, 2023.

[2] https://github.com/ddobric/neocortexapi, "NeoCortexApi," [Online].

[3] A. L. S. P. Z. A. Subutai Ahmad, "Unsupervised real-time anomaly detection for streaming data," *Science Direct,* vol. 262, no. ISSN 0925-2312, pp. 134-147, 2017.

[4] A. Barua, D. Muthirayan, P. P. Khargonekar and M. A. A. Faruque, "Hierarchical Temporal Memory Based Machine Learning for Real-Time, Unsupervised Anomaly Detection in Smart Grid: WiP Abstract," *2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPS),* pp. 188-189, 2020.